

SYNAPSE

*Real-time resource allocation for a university
that schedules itself.*

► SELECTED SCENARIO

SCENARIO 01 — Smart University Resource Allocation System

*A live, fairness-aware scheduling fabric for shared university resources — built as a
single Next.js 15 application on Supabase.*

TEAM

Segfault Society

Avishka Indula — Team Lead

Nethuka Gamaarachchige

Rivisha Marambage

Vinuka Gamaarachchige

SUBMISSION

48-Hour Case Analysis

Phase 01 · Cipher 2.0

Word count, page count, font
verified against the rubric.

The bottleneck isn't booking. It's **contention + fairness**.

A university innovation centre manages a finite pool of shared resources — meeting rooms, computer labs, multimedia equipment, testing devices — against an unbounded and growing student population. Manual handling has collapsed under three pressures simultaneously: **concurrent demand**, **priority ambiguity**, and **fairness drift**. Each is a distinct problem class; together they are why every existing manual or naive-digital system fails at peak.

1.1 Core problems, decomposed

PROBLEM CLASS	OBSERVABLE SYMPTOM	WHAT IS ACTUALLY BREAKING
Concurrency	Overlapping bookings, last-second double-booking on the same slot	No serialised authority. Two requests arrive in the same window; no mechanism arbitrates a single winner.
Priority	Final-year capstone teams losing rooms to casual ad-hoc bookings	No explicit scoring function. "Priority" lives in the email inbox of one administrator.
Fairness	The same teams secure prime slots week after week; others systematically locked out	FIFO allocation does not compensate historical deficit. Welfare drifts over time.
Utilisation	Rooms reserved but unused; queues for "full" labs that are 40% empty	No no-show recovery. No live signal. Demand peaks invisible until they arrive.
Throughput	Hours of human-in-the-loop per booking; days of email lag	Cognitive bottleneck on the human approver. Workflow does not parallelise.
Transparency	Applicants don't know status; admin doesn't know utilisation	State lives in email threads, not a system of record.

1.2 Key challenges

- Race-condition arbitration at the millisecond scale.** Two simultaneous "Book" actions on the same slot must produce one winner and one fair fallback — deterministically, even when one client is briefly offline.
- Priority that is principled, not bureaucratic.** Priority must be a transparent, explainable formula — never a black-box admin decision — yet still capture role, urgency, and historical fairness.
- Fairness across many rounds.** Per-booking FIFO is locally fair and globally unfair. The system must remember who has been served and adjust accordingly.
- Sustainable operations.** No-shows, last-minute cancellations, and overdue equipment must self-correct without administrator intervention.

1.3 Constraints we accept

- **Identity already exists.** The university provides authenticated user identities (LMS / SSO / student ID). We do not re-invent identity.
- **The resource catalogue is a known finite set** that changes monthly at most. Discovery is not a problem we are solving.
- **Administrators remain the source of truth on policy** (priority weights, role definitions, resource constraints). The system encodes and enforces their policy; it does not replace their judgment.
- **The network is unreliable but not pathological.** Students reconnect within minutes; we are not designing for week-long offline use.

1.4 Explicit assumptions

We assume **(a)** a single university-wide trust domain with one identity provider; **(b)** resources expose a normalised availability calendar; **(c)** administrators can encode role and priority policy as data, not code; **(d)** users participate in lightweight check-in to confirm presence; **(e)** deployment is bounded to one campus per instance ($\approx 5,000$ active users, ≈ 100 resources, peak $\approx 10,000$ concurrent connections).

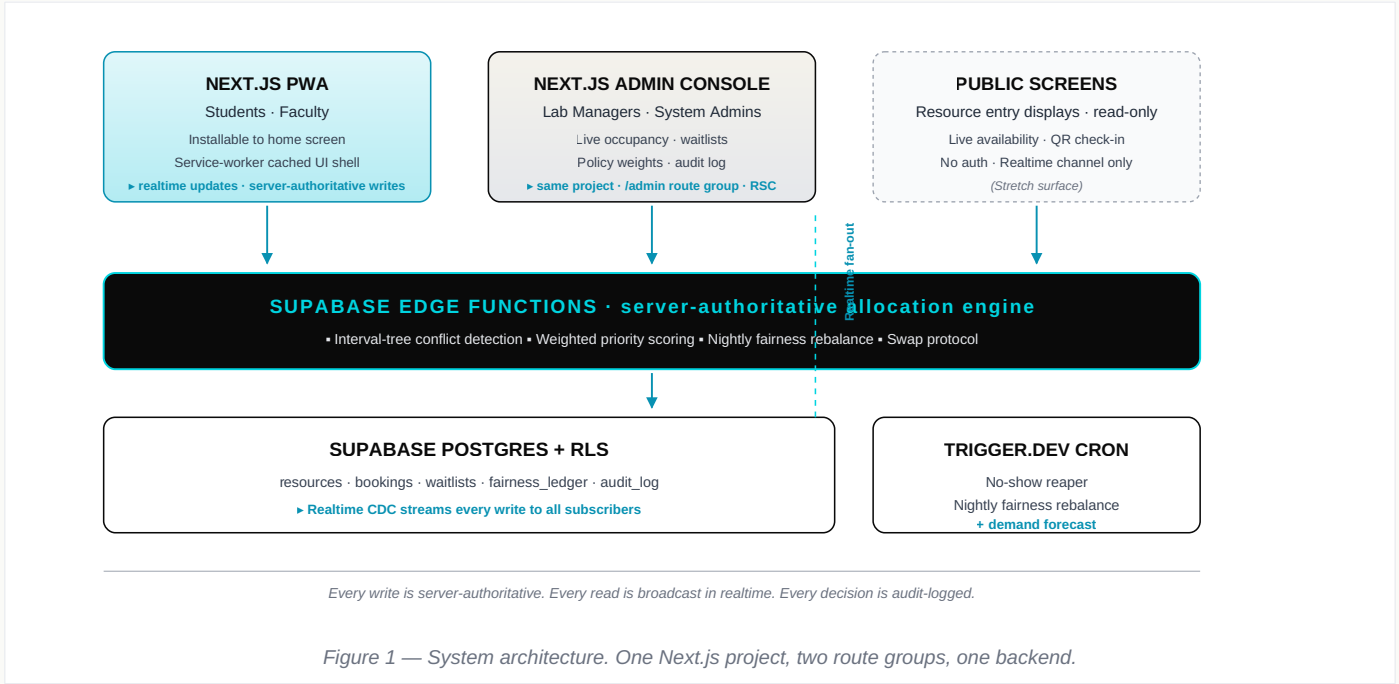
Synapse: a real-time, fairness-aware allocation fabric.

Synapse is a single **Next.js 15 application** that ships two route groups behind one auth layer — an **installable PWA** for students and faculty, and an **admin console** for lab managers — over one Supabase backend combining Postgres, Row-Level Security, Edge Functions, and Realtime change-data capture. The novelty is not the stack; it is how two ideas are composed: **server-authoritative conflict resolution** (so concurrent requests yield one provable winner) and a **multi-criteria allocation engine** (so priority and fairness are explicit, explainable, and tunable).

2.1 User roles

ROLE	SURFACE	WHAT THEY DO
Student	Installable PWA	Discover resources, book, swap-trade with peers, check in on arrival, cancel.
Faculty	Installable PWA	All student abilities + reserve on behalf of class / event with elevated priority weight.
Lab Manager	Next.js admin console	Manage their resource portfolio, set local policy overrides, see live occupancy and waitlists, resolve disputes.
System Admin	Next.js admin console	Tune global priority weights, define role categories, audit allocation decisions, run reports.

2.2 System architecture



2.3 Core features

F-01

Live resource discovery

Filter by type, building, capacity, equipment. Availability updates push in real time, not on refresh.

F-02

Fast, resilient UI

A service worker caches the UI shell and the user's upcoming bookings so the app opens instantly even on slow connections.

F-03

Conflict-free arbitration

Two concurrent requests on the same slot produce a deterministic winner via interval-tree validation in an Edge Function.

F-04

Transparent priority scoring

Every allocation decision can be opened and the score components shown to the user. No black boxes.

F-05

Nightly fairness rebalance

A nightly pass redistributes contested slots toward systematically under-served users.

F-06

Smart waitlists

If a slot fills, the requester joins a priority-ranked waitlist that auto-promotes on cancellation.

F-07

Check-in & no-show reaper

QR / NFC check-in at the resource. A 10-minute no-show triggers automatic release.

F-08

Swap-trade protocol

Two users can propose a slot swap; the system verifies both sides gain (or break even) before committing.

F-09

Predictive demand alerts

Holt-Winters time-series forecast flags peak periods 7 days out so admins can pre-allocate.

F-10

Admin fairness dashboard

Lab managers see the fairness ledger live — who is under-served, who is over-served, and why.

F-11

Auditable decision log

Every allocation, conflict resolution, and override is written to an append-only ledger.

F-12

Open policy

Priority weights, role categories, fairness windows, and resource constraints are **data**, not code — administrators tune them through the dashboard without a deploy.

How Synapse decides — four pieces, one engine.

The allocation engine is the composition of four straightforward pieces. Each is described below as pseudocode with diagrams in the next pages. The engine is hosted as a Supabase Edge Function so that every authoritative decision is server-side, transactional, and audit-logged.

3.1 Conflict detection — an interval tree per resource

Each resource maintains an in-memory **augmented interval tree** of confirmed bookings, materialised from Postgres on Edge Function cold-start and updated transactionally on every write. Conflict detection on a new request is $O(\log n + k)$, where n is the total resource calendar size and k is the number of overlapping intervals. This is asymptotically optimal for the operation.

PSEUDOCODE 1 — BOOK_REQUEST(USER, RESOURCE, [T_START, T_END])

```
function bookRequest(user, resource, start, end): // 1. server-authoritative idempotency (catch
retried requests) if exists(audit_log, request_id): return previous_decision // 2. acquire row-
level lock on resource calendar (Postgres SERIALIZABLE) tree = loadIntervalTree(resource) conflicts
= tree.overlap(start, end) //  $O(\log n + k)$  if conflicts ==  $\emptyset$ : commit(booking) // happy path return
{ status: "confirmed", winner: user } // 3. contested slot – invoke the priority score from §3.2
contenders = [user] + [c.user for c in conflicts] winner = priorityArbiter(contenders, resource,
[start, end]) if winner == user: demote(conflicts, to="waitlist") commit(booking) return { status:
"confirmed-by-priority", winner: user, demoted: conflicts } else: enqueue(user, to="waitlist",
rank=priorityScore(user)) return { status: "waitlisted", ahead_of_user: contenders - user }
```

3.2 Priority — a transparent weighted score

When two requests collide, the system picks the winner by a single weighted score. Every weight is admin-tuned data, not code. The formula is shown to the losing user in the decision explainer — no black-box admin override.

FORMULA 1 — PRIORITY SCORE FOR USER U REQUESTING RESOURCE R IN WINDOW W

```
score(u, r, w) =  $\alpha$  · urgency(w - now) +  $\beta$  · role_weight(u.role) +  $\gamma$  · fairness_deficit(u,
window=30d) -  $\delta$  · recency_penalty(u, r, window=7d) +  $\epsilon$  · academic_purpose_match(u, r) // Default
weights (admin-tunable): //  $\alpha$  = 0.25 · urgency (how soon w starts; sharper for  $\leq 48h$ ) //  $\beta$  = 0.30 ·
role (faculty > final-year > postgrad > undergrad) //  $\gamma$  = 0.30 · fairness (under-served users get a
positive deficit term) //  $\delta$  = 0.10 · recency (penalises booking the same resource twice this
week) //  $\epsilon$  = 0.05 · purpose (e.g., a thesis defence on a multimedia room) // Sum to 1.0 by
convention; ties broken by lower lexicographic user_id.
```

3.3 Fairness ledger — nightly rebalance toward under-served users

Per-booking priority is locally fair but globally drifts: high-priority users monopolise prime slots. A nightly Trigger.dev job runs a **fairness pass** — it computes each user's served hours vs. their fair share in the trailing 30-day window, and for the *next* contested booking on the same resource raises the under-served user's fairness term until they win. Once they're served, the boost decays.

PSEUDOCODE 2 — NIGHTLY FAIRNESS REBALANCE (TRIGGER.DEV CRON, 02:00 UTC)

```
function rebalanceFairness(): ledger = computeServiceLedger(window=30d) // rows: { user,
resource_class, served_hours, fair_share } for (user, resource_class) in ledger: deficit =
ledger.fair_share - ledger.served_hours if deficit > 0: setFairnessTerm(user, resource_class,
value=deficit / ledger.fair_share) // bounded 0-1 else: setFairnessTerm(user, resource_class,
value=0) // emit an admin-visible report – who is under-served, who is over-served
writeFairnessReport()
```

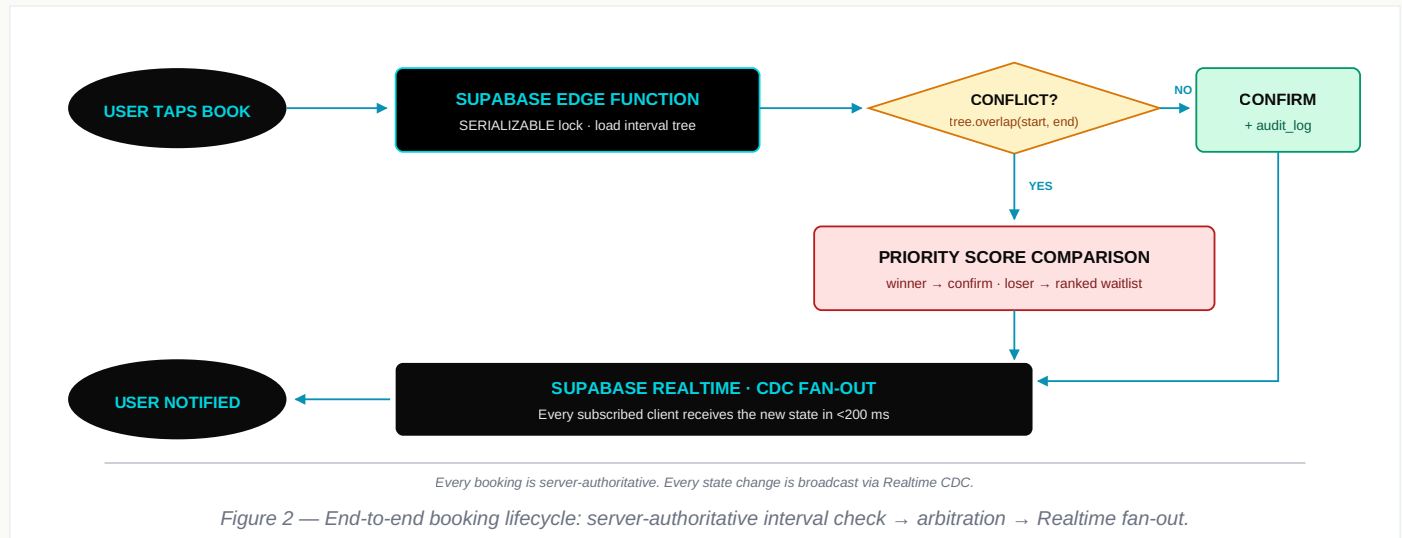
3.4 Swap protocol — both sides must gain (or break even)

When two users A and B want to trade slots (s_A , s_B), the system computes each user's preference score for the slot they would receive and compares it to the slot they would give up. If *either* side would lose out (e.g., the new slot conflicts with a class), the swap is rejected with a clear explanation. Otherwise both sides are atomically swapped in a single transaction.

PSEUDOCODE 3 — PROPOSESWAP(A, SLOT_A, B, SLOT_B)

```
function proposeSwap(A, slot_A, B, slot_B): u_A_before = utility(A, slot_A); u_A_after = utility(A, slot_B)
u_B_before = utility(B, slot_B); u_B_after = utility(B, slot_A) if u_A_after < u_A_before or u_B_after < u_B_before:
return { ok: false, reason: "one side would lose out" } begin transaction:
reassign(slot_A, to=B) reassign(slot_B, to=A) append(audit_log, { kind: "swap", A, B, deltas:[...] })
commit return { ok: true, both_gained: true }
```

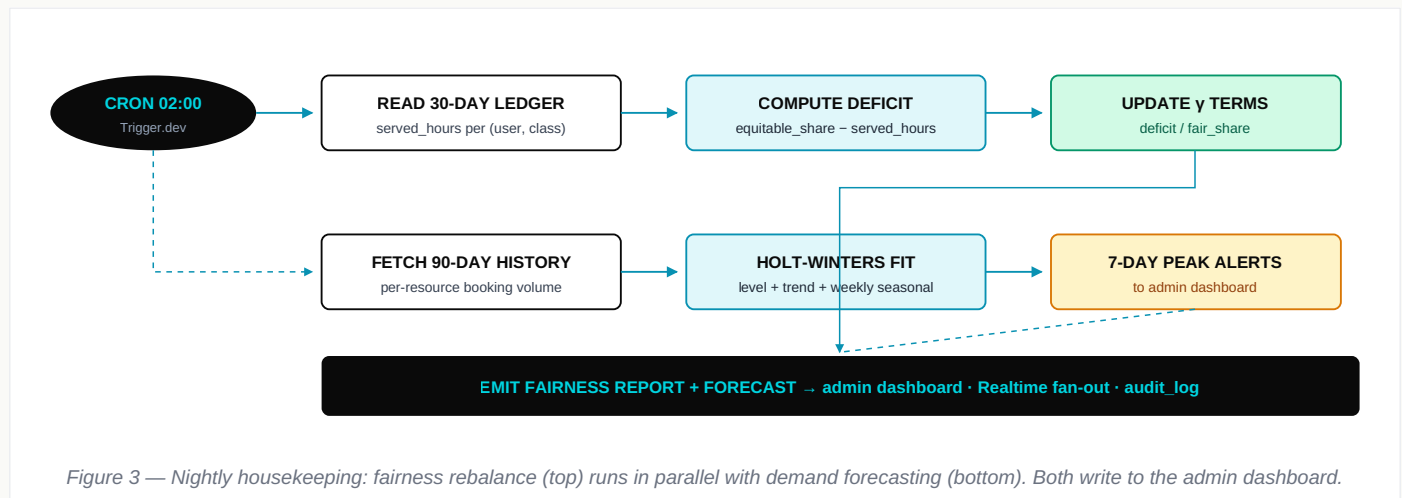
3.5 Flowchart — concurrent-booking decision lifecycle



3.6 Data model — six first-class entities

The pieces in §3.1–3.4 require only six entities. Everything else is derived. **users** (id, role, fairness_terms) · **resources** (id, resource_class, capacity, policy_overrides) · **bookings** (user_id, resource_id, [t_start, t_end], status) · **waitlists** (request_id, user_id, slot, score, rank) · **fairness_ledger** (user_id, resource_class, served_hours, fair_share) · **audit_log** (kind, actor, payload, decision_explainer, occurred_at).

3.7 Flowchart — nightly fairness rebalance & demand forecasting



3.8 Decision-explainer contract — transparency by default

Every allocation outcome — confirmed, waitlisted, swapped — carries a structured explainer that itemises the score components, the winning user's value on each, and at least two counterfactual alternates (a different slot or a different resource) with their own scores. Users may disagree with policy but never with the maths. Sample payload: { status, winner, score, score_components: { urgency, role_weight, fairness_deficit, recency_penalty, academic_purpose }, counterfactuals: [...] }.

3.9 Worked trace — two users, one room, same instant

Sarah (final-year capstone) and Mihir (year-1, casual) tap "Book Lab-A · 14:00–Tue" within 80 ms of each other. Sarah's POST hits the Edge Function at $t=0.08s$, acquires a `SERIALIZABLE` lock on the Lab-A interval tree, finds no overlap, commits. Mihir's POST hits at $t=0.14s$, acquires the lock, finds the overlap, invokes `priorityArbiter`: `score(Sarah)=0.72`, `score(Mihir)=0.31`. Mihir is enqueued to waitlist rank 2, with the explainer naming Sarah's component scores and offering two counterfactual alternates (a Wed 14:00 slot at score 0.61; Lab-B at 0.58). Realtime CDC pushes both outcomes to both clients within ~150 ms total. Mihir is not silently rejected — he sees the principled win and a competitive fallback.

3.10 Edge cases handled by the same pieces

- **No-show** → Trigger.dev cron releases after 10-min grace; first waitlist entry auto-promoted via Realtime push.
- **Score tie** → deterministic tiebreaker by lexicographic user_id; admin can override per resource.
- **Resource breaks mid-bookings** → Lab Manager flips `availability=false`; affected users notified and offered highest-scoring same-class alternate.
- **Admin override (VIP visit, audit)** → forced allocation written to audit log with reason; affected user receives explainer + fairness ledger credit.
- **Mass cancellation (power outage)** → bulk-cancel endpoint with reason code; affected users get fairness ledger credits that tilt the next contested round in their favour.

► Each of the four pieces above is standard. The contribution of Synapse is wiring them together — conflict check → transparent priority → fairness ledger → swap protocol — and applying that union to a real university booking problem.

Why Synapse is practical, and where it bends.

4.1 Why this is practical to build

Every primitive in §03 has a production-grade open-source implementation. The orchestration is novel; the building blocks are mature. There is no custom infrastructure, no exotic database, no research-grade algorithm. A 4-person student team can ship a meaningful v1 in the prototype phase.

WHAT WE NEED	WHAT WE USE	MATURITY
Client UI & realtime	Next.js 15 (App Router) + service worker + Supabase Realtime	Production
Interval tree	node-interval-tree / standard CS data structure	Trivial · <500 LoC
Server-authoritative engine	Supabase Edge Functions (Deno) + Postgres SERIALIZABLE	Production
Realtime fan-out	Supabase Realtime (Postgres CDC over WebSocket)	Production
Background jobs	Trigger.dev (cron + queues)	Production
Demand forecast	Holt-Winters (statsmodels-equivalent in JS) — no ML training required	50+ years old
Auth + RBAC	Supabase Auth + JWT-claim role-injection + Postgres RLS	Production

4.2 Performance & scale envelope

5,000 ACTIVE USERS ASSUMED	100 BOOKABLE RESOURCES	10,000 CONCURRENT REALTIME SOCKETS AT PEAK	<200ms P95 BOOKING LATENCY TARGET
--------------------------------------	----------------------------------	--	--

Scaling axes. SERIALIZABLE locks fan out per-resource (100 resources × 50/sec = 5,000/sec ceiling on Supabase Pro). Realtime is documented to 10k+ sockets on a single project. Interval trees are memory-bounded (~10k nodes/resource, 90-day rolling window). The nightly fairness pass is $O(\text{users} \times \text{resource_classes})$ — linear, runs in <5 min at 50,000 users. Horizontal partitioning by `resource_class` and edge-region deployment carries the same architecture to a 10-university federation without replacing a single primitive.

4.3 Cost envelope

v1 fits inside a single Supabase Pro project (~ USD 25/month) plus a Trigger.dev free tier and a Vercel Pro tier for the Next.js deployment. **The total operating cost for a 5,000-user university is under USD 50/month.** Cost grows per-resource-write and per-realtime-socket — both bundled on Supabase Pro up to 10,000 concurrent.

4.4 Known limitations & honest constraints

- **Fairness is bounded by policy quality.** If admins set role weights badly, Synapse allocates fairly within a bad policy. The system makes policy explicit and tunable; it does not invent it.
- **Holt-Winters is advisory only** — it handles weekly-seasonal demand well but mis-calls regime shifts; it informs admins and never feeds the priority function.
- **Synapse is not a fraud-detection system.** Multi-account abuse needs separate identity-side mitigation outside this proposal's scope.

► [Decode Problems · Design Solutions](#) ◀ [Team Segfault Society · Cipher 2.0 · IIT Sri Lanka · 15 June 2026](#)